format, if at all. When an error occurred in your app and you were trying to diagnose it, this inconsistency made it harder to connect the dots in your app to get the full picture and understand the problem.

Luckily, ASP.NET Core includes a new generic logging interface that you can plug into. It's used throughout the ASP.NET Core framework code itself, as well as by third-party libraries, and you can easily use it to create logs in your own code. With the ASP.NET Core logging framework, you can control the verbosity of logs coming from each part of your code, including the framework and libraries, and you can write the log output to any destination that plugs into the framework.

In this chapter I cover the .NET logging framework ASP.NET Core uses in detail, and I explain how you can use it to record events and diagnose errors in your own apps. In section 26.1 I'll describe the architecture of the logging framework. You'll learn how dependency injection (DI) makes it easy for both libraries and apps to create log messages, as well as to write those logs to multiple destinations.

In section 26.2 you'll learn how to write your own log messages in your apps with the `ILogger` interface. We'll break down the anatomy of a typical log record and look at its properties, such as the log level, category, and message.

Writing logs is useful only if you can read them, so in section 26.3 you'll learn how to add logging providers to your application. *Logging providers* control where your app writes your log messages, such as to the console, to a file, or even to an external service.

Logging is an important part of any application, but determining how much logging is enough can be a tricky question. On one hand, you want to provide sufficient information to be able to diagnose any problems. On the